

BÀI THỰC HÀNH CHƯƠNG 4

Lập trình CSDL với Spring Data JPA (Cơ bản)

Học phần: Lập trình ứng dụng với Java (14113014)

Chương 4: Lập trình CSDL với Spring Data JPA (Cơ bản)

Thời lượng: 3 tiết thực hành

1. Mục tiêu bài học

Sau khi hoàn thành bài thực hành này, sinh viên sẽ có khả năng:

- **Kiến thức:**
 - Hiểu rõ sự khác biệt giữa JDBC truyền thống và ORM.
 - Nắm vững kiến trúc phân tầng kết nối CSDL: Spring Data JPA → JPA → Hibernate → JDBC.
 - Hiểu vai trò của từng Annotation ánh xạ Entity cơ bản.
- **Kỹ năng:**
 - Cấu hình kết nối cơ sở dữ liệu MySQL trong tệp `application.properties`.
 - Thiết kế và khai báo Entity class chuẩn JPA sử dụng các annotation `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`.
 - Tạo Repository interface kế thừa `JpaRepository` để thực hiện các thao tác CRUD cơ bản mà không cần viết SQL.
 - Kiểm tra kết quả bằng cách quan sát bảng dữ liệu được tự động sinh ra và log SQL trên Console.
 - Viết REST API kết hợp với Spring Data JPA để thao tác dữ liệu qua HTTP.
- **Thái độ:** Nhận thức được lợi ích vượt trội của ORM so với JDBC truyền thống, đồng thời hiểu rõ nguyên tắc an toàn dữ liệu khi sử dụng cấu hình `ddl-auto` trong các môi trường khác nhau.

2. Chuẩn bị môi trường

Để thực hiện bài thực hành này, sinh viên cần chuẩn bị đầy đủ các công cụ sau:

- **JDK (Java Development Kit):** Phiên bản 17 hoặc 21 (LTS).
- **IDE:** IntelliJ IDEA Community Edition hoặc Visual Studio Code.

- **Hệ quản trị CSDL:** MySQL Server phiên bản 8.x đã cài đặt và đang chạy trên máy tính.
- **Công cụ quản trị CSDL:** MySQL Workbench, DBeaver, hoặc phpMyAdmin để quan sát dữ liệu trực tiếp trong database.
- **REST Client:** Postman hoặc công cụ dòng lệnh **curl** để kiểm thử các API.
- **Trình duyệt Web:** Google Chrome, Microsoft Edge hoặc Mozilla Firefox.

3. Kiến thức nền tảng

3.1. Tại sao không dùng JDBC trực tiếp?

Trong lập trình Java truyền thống, JDBC (Java Database Connectivity) là API tiêu chuẩn để kết nối đến cơ sở dữ liệu. Tuy nhiên, JDBC yêu cầu lập trình viên phải thực hiện rất nhiều công việc thủ công.

Quan sát đoạn mã JDBC dưới đây để truy vấn danh sách sản phẩm:

```
// Mã nguồn JDBC truyền thống - dài dòng và dễ lỗi
try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
    PreparedStatement stmt = conn.prepareStatement("SELECT * FROM products");
    ResultSet rs = stmt.executeQuery()) {
    while (rs.next()) {
        Product p = new Product();
        p.setId(rs.getLong("id"));
        p.setName(rs.getString("product_name"));
        p.setPrice(rs.getBigDecimal("price"));
        // Phải map THỦ CÔNG từng cột sang từng thuộc tính...
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

Các hạn chế của JDBC

- Mã nguồn lặp lại rất nhiều (boilerplate code), chiếm tới 70-80% tổng code kết nối.
- SQL viết dạng String nên IDE không thể kiểm tra lỗi chính tả tên bảng hoặc tên cột lúc viết code.
- Khi đổi tên cột dưới database, lập trình viên phải tìm và sửa tất cả các String SQL liên quan trong toàn bộ dự án.

3.2. ORM và kiến trúc kết nối trong Spring Boot

ORM (Object-Relational Mapping) giải quyết các hạn chế trên bằng cách **tự động ánh xạ** giữa đối tượng Java và bảng CSDL:

- **Class Java** ↔ **Bảng (Table)** trong DB
- **Thuộc tính (Attribute)** ↔ **Cột (Column)**
- **Thể hiện đối tượng (Object Instance)** ↔ **Hàng (Row)**
- **Kiểu dữ liệu Java** ↔ **Kiểu dữ liệu CSDL** (ví dụ: **String** → **VARCHAR**)

Kiến trúc kết nối CSDL trong Spring Boot được tổ chức theo 4 tầng từ cao xuống thấp:

Tầng	Công nghệ	Vai trò
1 (Cao nhất)	Spring Data JPA	Cung cấp Repository interface tiện lợi, tự động sinh code CRUD
2	JPA (Jakarta Persistence API)	Đặc tả tiêu chuẩn, định nghĩa các Annotation và Interface
3	Hibernate	ORM Provider, triển khai thực tế đặc tả JPA, sinh câu SQL
4 (Thấp nhất)	JDBC	Kết nối vật lý trực tiếp đến Database

Chú ý

Spring Data JPA không thay thế Hibernate hay JDBC. Nó chỉ là tầng trừu tượng hóa ở trên cùng, giúp lập trình viên viết ít code hơn. Bên dưới, Hibernate vẫn sử dụng JDBC để kết nối thực sự đến database.

4. Nội dung thực hành

Bài 1: Khởi tạo dự án Spring Boot với Spring Data JPA

Bước 1: Tạo dự án mới trên Spring Initializr

Truy cập trang web Spring Initializr (<https://start.spring.io>) hoặc sử dụng giao diện IntelliJ IDEA để khởi tạo dự án với các thông tin sau:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** Phiên bản 3.x.x (ví dụ: **3.5.16** hoặc bản Stable mới nhất)
- **Project Metadata:**
 - **Group:** **vn.edu.gdu**
 - **Artifact:** **ch4-lab-spring-jpa**
 - **Name:** **ch4-lab-spring-jpa**
 - **Description:** Spring Boot Data JPA Lab Project for GDU Students
 - **Package name:** **vn.edu.gdu.springjpalab**
 - **Packaging:** Jar
 - **Configuration:** Properties

- **Java:** 17 hoặc 21
- **Dependencies:** Thêm các dependency sau:
 - **Spring Web:** Hỗ trợ xây dựng REST API, ứng dụng dùng Spring MVC và Tomcat.
 - **Spring Data JPA:** Module kết nối CSDL thông qua JPA/Hibernate.
 - **MySQL Driver:** Trình điều khiển kết nối tới hệ CSDL MySQL.
 - **Lombok:** Thư viện tiện ích tự động sinh Getter, Setter, Constructor.

Nhấn **GENERATE** để tạo và tải về dự án (dưới dạng file .zip).

Bước 2: Nhập dự án vào IDE và kiểm tra cấu trúc

Sau khi giải nén và mở dự án trong IntelliJ IDEA, sinh viên cần đợi Maven đồng bộ dependencies. Đảm bảo cấu trúc thư mục của dự án tuân thủ cấu trúc chuẩn:

```
ch4-lab-spring-jpa
├── pom.xml
├── src
│   └── main
│       ├── java
│       │   └── vn.edu.gdu.springjpalab
│       │       └── SpringJpaLabApplication.java
│       └── resources
│           ├── application.properties
│           ├── static
│           └── templates
```

Bước 3: Kiểm tra file pom.xml

Mở file **pom.xml** và đảm bảo đã khai báo đầy đủ các dependencies cần thiết:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
```

```

</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

```

Bước 4: Tạo cơ sở dữ liệu MySQL

Mở MySQL Workbench hoặc sử dụng dòng lệnh MySQL để tạo cơ sở dữ liệu mới cho bài thực hành:

```

-- Tạo database mới cho bài thực hành
CREATE DATABASE IF NOT EXISTS gdu_jpa_lab
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Kiểm tra database đã tạo thành công
SHOW DATABASES LIKE 'gdu_jpa_lab';

```

Ghi chú

Bộ ký tự `utf8mb4` được khuyến nghị sử dụng thay cho `utf8` vì nó hỗ trợ đầy đủ các ký tự Unicode bao gồm cả emoji, trong khi `utf8` của MySQL (thực chất là `utf8mb3`) chỉ hỗ trợ tối đa 3 byte cho mỗi ký tự.

Bài 2: Cấu hình kết nối Database trong application.properties

Mục tiêu:

Cấu hình Spring Boot kết nối đến cơ sở dữ liệu MySQL vừa tạo ở Bài 1, đồng thời thiết lập các tham số Hibernate để hỗ trợ quá trình phát triển.

Bước 1: Cấu hình DataSource

Mở tệp `src/main/resources/application.properties` và thêm các cấu hình kết nối sau:

```

# =====
# CẤU HÌNH KẾT NỐI CSDL (DataSource)
# =====

# 1. URL kết nối JDBC đến MySQL
#   - localhost:3306 là địa chỉ và cổng mặc định của MySQL Server
#   - gdu_jpa_lab là tên database vừa tạo
spring.datasource.url=jdbc:mysql://localhost:3306/gdu_jpa_lab?

```

```
allowPublicKeyRetrieval=true&useSSL=false&serverTimezone=Asia/Ho_Chi_Minh

# 2. Tài khoản đăng nhập MySQL
spring.datasource.username=root
spring.datasource.password=your_password

# 3. Driver kết nối MySQL (có thể bỏ qua ở Spring Boot 3.x vì tự phát hiện)
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

! Quan trọng

Sinh viên cần thay thế giá trị `your\_password` bằng mật khẩu MySQL thực tế trên máy tính của mình. Nếu tài khoản MySQL không phải `root`, hãy thay đổi giá trị `spring.datasource.username` cho phù hợp.

Bước 2: Cấu hình hành vi JPA/Hibernate

Tiếp tục thêm các dòng cấu hình sau vào tệp `application.properties`:

```
# =====
# CẤU HÌNH JPA / HIBERNATE
# =====

# 4. Hiển thị câu lệnh SQL được Hibernate tự động sinh ra trên console
spring.jpa.show-sql=true

# 5. Định dạng câu lệnh SQL cho dễ đọc (xuống dòng, thụt lề)
spring.jpa.properties.hibernate.format_sql=true

# 6. Chỉ định Dialect của MySQL 8 để Hibernate sinh SQL đúng cú pháp
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

# 7. Chế độ tự động sinh/cập nhật bảng từ Entity Class
# - update: Cập nhật bảng theo Entity, giữ nguyên dữ liệu cũ (dùng cho Development)
spring.jpa.hibernate.ddl-auto=update
```

Bước 3: Hiểu rõ các chế độ "ddl-auto"

Giá trị	Hành vi	Môi trường phù hợp
<code>none</code>	Không thay đổi gì dưới database	Production (An toàn nhất)
<code>validate</code>	Chỉ kiểm tra cấu trúc bảng có khớp Entity không	Production
<code>update</code>	Cập nhật bảng cũ, thêm cột mới nếu có, giữ dữ liệu	Development
<code>create</code>	Xóa bảng cũ + tạo bảng mới mỗi lần chạy ứng dụng	Testing (Mất dữ liệu!)
<code>create-drop</code>	Giống <code>create</code> , thêm DROP khi tắt ứng dụng	Unit Test

⚠ Lưu ý Nguyên tắc an toàn dữ liệu:

Tuyệt đối **KHÔNG** sử dụng ``update``, ``create`` hoặc ``create-drop`` trong môi trường **Production**. Chỉ sử dụng ``none`` hoặc ``validate`` khi triển khai sản phẩm thực tế. Việc sơ ý sử dụng ``create`` trên server vận hành có thể gây mất sạch toàn bộ dữ liệu khách hàng.

Bước 4: Chạy thử ứng dụng để kiểm tra kết nối

Nhấn nút **Run** trong IntelliJ IDEA để khởi động ứng dụng Spring Boot. Quan sát log console:

- **Thành công:** Xuất hiện dòng `Started Ch4LabSpringJpaApplication in X seconds` và không có lỗi đỏ.
- **Các lỗi phổ biến:**
 - `Access denied for user 'root'@'localhost': Sai tên đăng nhập hoặc mật khẩu MySQL.`
 - `Unknown database 'gdu_jpa_lab': Chưa tạo database ở Bước 4 của Bài 1.`
 - `Communications link failure: MySQL Server chưa được khởi động.`

Bài 3: Thiết kế Entity Class đầu tiên - Product

Mục tiêu:

Tạo một lớp Entity Java đại diện cho bảng `products` trong cơ sở dữ liệu, sử dụng các annotation chuẩn JPA.

Bước 1: Tạo package ``entity``

Trong package gốc `vn.edu.gdu.springjpalab`, tạo một package con mới có tên `entity`.

Bước 2: Tạo lớp Entity Product

Tạo tệp `Product.java` bên trong package `entity` với nội dung đầy đủ như sau:

```
package vn.edu.gdu.springjpalab.entity;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity                                // (1) Đánh dấu đây là Entity JPA
@Table(name = "products")             // (2) Ánh xạ đến bảng "products" trong DB
public class Product {

    @Id                                // (3) Đánh dấu trường này là khóa chính
    @GeneratedValue(strategy = GenerationType.IDENTITY) // (4) Tự động tăng
    private Long id;

    @Column(name = "product_name", nullable = false, length = 150)
    private String name;                // (5) Ánh xạ đến cột "product_name"
```

```

@Column(name = "price", nullable = false, precision = 10, scale = 2)
private BigDecimal price;    // (6) Kiểu tiền tệ dùng BigDecimal

@Column(name = "sku", unique = true, nullable = false, length = 50)
private String sku;          // (7) Mã sản phẩm - duy nhất

@Column(name = "created_at")
private LocalDateTime createdAt;

// — 1. Constructor không tham số (BẮT BUỘC cho JPA) —
protected Product() {
}

// — 2. Constructor có tham số (tiện lợi cho lập trình viên) —
public Product(String name, BigDecimal price, String sku) {
    this.name = name;
    this.price = price;
    this.sku = sku;
    this.createdAt = LocalDateTime.now();
}

// — 3. Getter và Setter —
public Long getId() {
    return id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public BigDecimal getPrice() {
    return price;
}

public void setPrice(BigDecimal price) {
    this.price = price;
}

public String getSku() {
    return sku;
}

public void setSku(String sku) {
    this.sku = sku;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

@Override
public String toString() {
    return "Product{id=" + id + ", name='" + name + "', price=" + price +
        ", sku='" + sku + "', createdAt=" + createdAt + "}";
}
}

```


Bước 3: Phân tích chi tiết từng Annotation

A. `@Entity`

- Thuộc package `jakarta.persistence.Entity`.
- Báo cho Spring Boot và Hibernate biết lớp này là một thực thể CSDL cần được quản lý.
- **Quy tắc bắt buộc:** Lớp phải có constructor không tham số (no-arg constructor), không được khai báo `final`, và phải có trường `@Id`.

B. `@Table(name = "products")`

- Chỉ định tên bảng dưới database là `products`.
- Nếu không khai báo `@Table`, Hibernate mặc định sẽ lấy tên lớp (`Product`) làm tên bảng.
- **Best Practice:** Luôn đặt tên bảng ở dạng số nhiều, sử dụng `snake_case`.

C. `@Id` và `@GeneratedValue`

- `@Id`: Đánh dấu trường `id` là khóa chính (Primary Key).
- `@GeneratedValue(strategy = GenerationType.IDENTITY)`: Hibernate ủy quyền cho cột Auto-Increment của MySQL tự sinh giá trị.

D. `@Column`

- `name`: Tên cột thực tế trong database (nên dùng `snake_case`).
- `nullable = false`: Thiết lập ràng buộc NOT NULL.
- `unique = true`: Dữ liệu không được trùng lặp.
- `length`: Giới hạn số ký tự tối đa cho cột String.
- `precision` và `scale`: Cấu hình độ chính xác cho kiểu số thập phân.

! Quan trọng

Lưu ý quan trọng khi thiết kế Entity:

1. Constructor không tham số là bắt buộc vì Hibernate dùng cơ chế Reflection để khởi tạo đối tượng. Nếu thiếu, chương trình sẽ báo lỗi `InstantiationException`.
2. Tránh khai báo class là `final` vì Hibernate cần tạo proxy class để thực hiện Lazy Loading.
3. Đối với tiền tệ, luôn sử dụng `BigDecimal`, tuyệt đối không dùng `double` hay `float` do lỗi làm tròn nhị phân.

Bước 4: Chạy ứng dụng và kiểm tra bảng được tạo tự động

1. Nhấn **Run** để khởi động lại ứng dụng Spring Boot.
2. Quan sát log Console. Với cấu hình `ddl-auto=update` và `show-sql=true`, bạn sẽ thấy Hibernate tự động sinh câu SQL tương tự:

```
Hibernate:
    create table products (
        id bigint not null auto_increment,
```

```
        created_at datetime(6),
        product_name varchar(150) not null,
        price decimal(10,2) not null,
        sku varchar(50) not null,
        primary key (id)
    ) engine=InnoDB
```

Hibernate:

```
    alter table products
        add constraint UK_sku unique (sku)
```

3. Mở MySQL Workbench (hoặc DBeaver) và kiểm tra database `gdu_jpa_lab`. Bảng `products` sẽ xuất hiện với cấu trúc cột khớp với Entity class.

Chú ý

Nếu muốn Hibernate tạo lại bảng hoàn toàn mới (xóa bảng cũ), hãy tạm thay ``ddl-auto=update`` thành ``ddl-auto=create``. Nhớ đổi lại ``update`` sau khi kiểm tra xong để tránh mất dữ liệu.

Bài 4: Tạo Repository Interface cho Entity Product

Mục tiêu:

Khai báo Repository interface kế thừa `JpaRepository` để Spring Data JPA tự động cung cấp các phương thức CRUD cơ bản mà không cần viết bất kỳ dòng SQL nào.

Bước 1: Tạo package `repository`

Trong package gốc `vn.edu.gdu.springjpalab`, tạo package con mới có tên `repository`.

Bước 2: Tạo interface `ProductRepository`

Tạo tệp `ProductRepository.java` bên trong package `repository`:

```
package vn.edu.gdu.springjpalab.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import vn.edu.gdu.springjpalab.entity.Product;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    // Spring Data JPA tự động cung cấp các phương thức CRUD:
    // - save(entity)           : Thêm mới hoặc cập nhật
    // - findById(id)           : Tìm theo khóa chính
    // - findAll()              : Lấy tất cả bản ghi
    // - deleteById(id)         : Xóa theo khóa chính
    // - count()                : Đếm tổng số bản ghi
    // - existsById(id)         : Kiểm tra tồn tại
}
```

Ghi chú

`JpaRepository<Product, Long>` nhận hai tham số: Tham số thứ nhất là kiểu Entity (`Product`), tham số thứ hai là kiểu dữ liệu của khóa chính (`Long`). Spring Data JPA sẽ tự động tạo lớp triển khai (implementation class) cho interface này tại thời điểm runtime. Lập trình viên không cần viết bất kỳ dòng code SQL nào.

Bài 5: Xây dựng REST API thao tác dữ liệu Product

Mục tiêu:

Phát triển REST Controller kết hợp với **ProductRepository** để tạo API thêm mới, truy vấn, cập nhật và xóa sản phẩm.

Bước 1: Tạo package `controller`

Trong package gốc **vn.edu.gdu.springjpalab**, tạo package con mới có tên **controller**.

Bước 2: Tạo lớp ProductController

Tạo tệp **ProductController.java** bên trong package **controller**:

```
package vn.edu.gdu.springjpalab.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import vn.edu.gdu.springjpalab.entity.Product;
import vn.edu.gdu.springjpalab.repository.ProductRepository;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductRepository productRepository;

    // Constructor Injection (tiêm phụ thuộc qua Constructor)
    public ProductController(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
}
```

```

// — 1. API lấy danh sách tất cả sản phẩm —
@GetMapping
public List<Product> getAllProducts() {
    return productRepository.findAll();
}

// — 2. API lấy sản phẩm theo ID —
@GetMapping("/{id}")
public ResponseEntity<Product> getProductById(@PathVariable Long id) {
    Optional<Product> product = productRepository.findById(id);
    return product.map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

// — 3. API thêm mới sản phẩm —
@PostMapping
public ResponseEntity<Product> createProduct(@RequestBody Product product) {
    Product savedProduct = productRepository.save(product);
    return ResponseEntity.status(HttpStatus.CREATED).body(savedProduct);
}

// — 4. API cập nhật sản phẩm —
@PutMapping("/{id}")
public ResponseEntity<Product> updateProduct(
    @PathVariable Long id,
    @RequestBody Product productDetails) {
    Optional<Product> existingProduct = productRepository.findById(id);
    if (existingProduct.isPresent()) {
        Product product = existingProduct.get();
        product.setName(productDetails.getName());
        product.setPrice(productDetails.getPrice());
        product.setSku(productDetails.getSku());
        Product updatedProduct = productRepository.save(product);
        return ResponseEntity.ok(updatedProduct);
    }
    return ResponseEntity.notFound().build();
}

// — 5. API xóa sản phẩm —
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
    if (productRepository.existsById(id)) {
        productRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.notFound().build();
}

```

```
// — 6. API đếm tổng số sản phẩm —  
@GetMapping("/count")  
public long countProducts() {  
    return productRepository.count();  
}  
}
```

Bước 3: Kiểm tra cấu trúc dự án hoàn chỉnh

Đến bước này, cấu trúc package của dự án phải có dạng:

```
vn.edu.gdu.springjpalab  
├─ Ch4LabSpringJpaApplication.java  
├─ controller  
│   └─ ProductController.java  
├─ entity  
│   └─ Product.java  
└─ repository  
    └─ ProductRepository.java
```

Bài 6: Kiểm thử các API bằng Postman và cURL

Mục tiêu:

Sử dụng Postman và dòng lệnh cURL để kiểm thử toàn bộ các thao tác CRUD trên API vừa xây dựng, đồng thời quan sát các câu SQL được Hibernate sinh ra trên Console.

Bước 1: Khởi động ứng dụng

Nhấn **Run** để khởi động Spring Boot. Đảm bảo ứng dụng đã chạy thành công trên cổng 8080.

Bước 2: Thêm mới sản phẩm (POST)

Sử dụng Postman:

- Method: **POST**
- URL: **http://localhost:8080/api/products**
- Header: **Content-Type: application/json**
- Body (raw JSON):

```
{  
  "name": "Laptop Dell XPS 15",  
  "price": 35500000,  
}
```

```
"sku": "DELL-XPS15-001",
"createdAt": "2026-07-09T10:00:00"
}
```

Sử dụng cURL:

```
curl -X POST http://localhost:8080/api/products \
-H "Content-Type: application/json" \
-d '{
  "name": "Laptop Dell XPS 15",
  "price": 35500000,
  "sku": "DELL-XPS15-001",
  "createdAt": "2026-07-09T10:00:00"
}'
```

Kết quả mong đợi: Trả về HTTP 201 Created cùng với JSON chứa sản phẩm vừa thêm (bao gồm **id** được tự sinh).

Tiếp tục thêm thêm 2 sản phẩm nữa để có dữ liệu phong phú:

```
curl -X POST http://localhost:8080/api/products \
-H "Content-Type: application/json" \
-d '{"name": "iPhone 16 Pro Max", "price": 34990000, "sku": "APL-IP16PM-001",
"createdAt": "2026-07-09T10:05:00"}'
```

```
curl -X POST http://localhost:8080/api/products \
-H "Content-Type: application/json" \
-d '{"name": "Samsung Galaxy S25 Ultra", "price": 31990000, "sku": "SAM-
S25U-001", "createdAt": "2026-07-09T10:10:00"}'
```

Bước 3: Quan sát log SQL trên Console

Sau mỗi lần gọi API thêm sản phẩm, quan sát log Console. Hibernate sẽ hiển thị câu SQL INSERT tương ứng:

```
Hibernate:
insert
into
  products
(created_at, product_name, price, sku)
values
  (?, ?, ?, ?)
```

Bước 4: Truy vấn danh sách sản phẩm (GET)

Lấy tất cả sản phẩm:

```
curl http://localhost:8080/api/products
```

Lấy sản phẩm theo ID:

```
curl http://localhost:8080/api/products/1
```

Đếm tổng số sản phẩm:

```
curl http://localhost:8080/api/products/count
```

Bước 5: Cập nhật sản phẩm (PUT)

```
curl -X PUT http://localhost:8080/api/products/1 \
-H "Content-Type: application/json" \
-d '{
  "name": "Laptop Dell XPS 15 (2026 Edition)",
  "price": 37500000,
  "sku": "DELL-XPS15-002"
}'
```

Kết quả mong đợi: Trả về HTTP 200 OK cùng JSON chứa sản phẩm đã cập nhật.

Bước 6: Xóa sản phẩm (DELETE)

```
curl -X DELETE http://localhost:8080/api/products/3
```

Kết quả mong đợi: Trả về HTTP 204 No Content (xóa thành công, không có nội dung trả về).

Bước 7: Kiểm tra dữ liệu trong MySQL Workbench

Mở MySQL Workbench (hoặc DBeaver), kết nối đến database `gdu_jpa_lab` và chạy câu truy vấn:

```
SELECT * FROM products;
```

Xác nhận rằng dữ liệu trong bảng `products` phản ánh đúng các thao tác CRUD đã thực hiện qua API.



Chú ý

Hãy quan sát log SQL trên Console tại mỗi bước. So sánh câu SQL mà Hibernate sinh ra với thao tác API tương ứng. Điều này giúp sinh viên hiểu rõ cách ORM dịch các lời gọi Java thành câu SQL thực tế.

Bài 7: Thiết kế Entity thứ hai - Student (Bài tập tự luyện)

Mục tiêu:

Sinh viên tự thiết kế một Entity class mới từ đầu dựa trên kiến thức đã học ở các bài trước.

Yêu cầu:

Thiết kế Entity class **Student** đại diện cho bảng **students** với các thuộc tính sau:

Thuộc tính Java	Tên cột DB	Kiểu dữ liệu	Ràng buộc
id	id	Long	Khóa chính, tự động tăng
studentCode	student_code	String	Duy nhất, không null, tối đa 20 ký tự
fullName	full_name	String	Không null, tối đa 100 ký tự
email	email	String	Duy nhất, không null, tối đa 150 ký tự
gpa	gpa	BigDecimal	Có thể null, precision 3 scale 2
enrollmentDate	enrollment_date	LocalDate	Có thể null

Các bước thực hiện:

1. Tạo lớp **Student.java** trong package **entity** với đầy đủ các annotation JPA.
2. Tạo interface **StudentRepository.java** trong package **repository** kế thừa **JpaRepository<Student, Long>**.
3. Tạo lớp **StudentController.java** trong package **controller** với các API CRUD tương tự **ProductController**.
4. Chạy ứng dụng và kiểm tra bảng **students** được tự động tạo trong MySQL.
5. Sử dụng Postman hoặc cURL để thêm ít nhất 3 bản ghi sinh viên và truy vấn lại.

5. Câu hỏi ôn tập và củng cố kiến thức

Hãy ghi chép lại câu trả lời cho các câu hỏi sau vào báo cáo thực hành:

- 1. Phân biệt JDBC và ORM:** Em hãy liệt kê ít nhất 3 hạn chế cốt lõi của JDBC truyền thống mà ORM giúp giải quyết. Cho ví dụ minh họa cho mỗi hạn chế.
 - 2. Kiến trúc phân tầng:** Em hãy giải thích mối quan hệ giữa Spring Data JPA, JPA, Hibernate và JDBC. Tại sao Spring Data JPA không thể hoạt động nếu không có Hibernate?
 - 3. Constructor không tham số:** Tại sao Entity class bắt buộc phải có constructor không tham số? Điều gì xảy ra nếu lập trình viên quên khai báo constructor này?
 - 4. Chế độ ddl-auto:** Chế độ **ddl-auto** nào cực kỳ nguy hiểm trong môi trường Production? Giải thích tại sao và nêu chế độ phù hợp cho Production.
 - 5. Annotation @Column:** Em hãy giải thích ý nghĩa của từng thuộc tính: **name**, **nullable**, **unique**, **length**, **precision**, **scale** trong annotation **@Column**. Tại sao nên sử dụng **BigDecimal** thay vì **double** cho kiểu tiền tệ?
 - 6. Thực hành bổ sung:** Em hãy thêm một trường **description** (kiểu String, tối đa 500 ký tự, có thể null) vào Entity **Product**, sau đó chạy lại ứng dụng và quan sát câu SQL ALTER TABLE mà Hibernate sinh ra trên Console. Ghi lại câu SQL đó vào báo cáo.
-

6. Tài liệu tham khảo

Để nghiên cứu sâu hơn về Spring Data JPA và Hibernate, sinh viên tham khảo các nguồn tài liệu chính thống dưới đây:

1. Spring Data JPA Official Reference Guide:

- Link: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- Nội dung: Hướng dẫn chuẩn nhất về Repository, Query Methods, và các tính năng nâng cao.

2. Hibernate ORM User Guide:

- Link: <https://hibernate.org/orm/documentation/>
- Nội dung: Cơ chế quản lý Entity Lifecycle, Caching và tối ưu hiệu năng SQL.

3. Baeldung Spring Boot Persistence Guides:

- Link: <https://www.baeldung.com/category/persistence/spring-persistence>
- Nội dung: Các ví dụ thực hành trực quan và thực tế cho các tình huống lập trình CSDL.